

EOAT Command Center Improvement Roadmap

Purpose

This document organizes the proposed EOAT Command Center improvements into practical development phases. It is meant to guide Codex or another developer through the next major upgrade cycle without turning the app into one enormous Python junk drawer with feelings.

This roadmap does **not** deeply specify every individual feature. A separate feature-specific document should handle detailed UI layouts, field lists, algorithms, and exact behavior. This document focuses on:

- Which improvements belong together
- Why each phase exists
- What each phase should produce
- Dependencies between phases
- Acceptance criteria for each phase
- Testing and safety expectations

Feature #19, **Robot Info entity system**, is intentionally excluded because it is already covered elsewhere.

Roadmap Summary

Recommended Phase Order

1. **Phase 0 — Current-State Audit and Safety Baseline**
2. **Phase 1 — App Architecture Foundation**
3. **Phase 2 — Audit Workflow Stabilization**
4. **Phase 3 — Audit Coach and Guided Completion**
5. **Phase 4 — Annotations, Notes, Tags, and Open Items**
6. **Phase 5 — Workbook Validation and Repair System**
7. **Phase 6 — Dashboard Performance, Cache, and Telemetry**
8. **Phase 7 — Scheduled Reports and Report Reliability**
9. **Phase 8 — Search, Navigation, and Productivity Layer**
10. **Phase 9 — Evidence, Photos, PM, BOM, and Documentation Coverage**
11. **Phase 10 — Standards, FMEA, Pilot, and Engineering Analysis**
12. **Phase 11 — Final Handoff and Leadership-Ready Outputs**
13. **Phase 12 — Repo Safety, Release Readiness, and Long-Term Maintenance**

This order is intentional. The first phases stabilize the app skeleton, then improve auditing, then unify open issues, then improve validation and reporting, then add richer engineering outputs.

Do **not** start with the flashiest UI ideas. That is how apps become beautiful coffins for bad architecture.

Feature Allocation Map

Feature #	Feature Name	Roadmap Phase
1	Audit Coach Mode	Phase 3
2	Apply/reject annotation suggestions	Phase 4
3	Unified Open Items / Action Board	Phase 4
4	Finish This Audit button	Phase 3
5	Dirty-form protection	Phase 2
6	Split AuditPage into smaller pieces	Phase 2
7	Configurable defaults	Phase 2
8	Gripper presets mini parts database	Phase 9
9	Repairable validation findings	Phase 5
10	“Why is this field hidden?” explanations	Phase 3
11	Global command palette	Phase 8
12	Global search	Phase 8
13	Cache staleness improvements	Phase 6
14	Structured performance logging	Phase 6
15	Scheduled report calendar preview	Phase 7
16	Catch-up summaries	Phase 7
17	Smarter scheduled report content	Phase 7
18	Task Scheduler preflight	Phase 7
19	Robot Info entity system	Excluded, covered elsewhere
20	Field history and audit revision history	Phase 2 / Phase 5
21	Backup retention cleanup	Phase 12
22	Local SQLite audit mirror	Phase 5 / Phase 6
23	SQLite FTS search	Phase 8
24	Photo evidence coverage	Phase 9
25	Phone-friendly photo intake	Phase 9
26	Pilot candidate evidence packet	Phase 10 / Phase 11

Feature #	Feature Name	Roadmap Phase
27	FMEA generation from tags and issues	Phase 10
28	Standards compliance score	Phase 10
29	Compatibility impact preview	Phase 2
30	Physical vs compatible row consistency checks	Phase 5
31	App-native Press View	Phase 8 / Phase 10
32	Workbook lock detector	Phase 5
33	Validation severity levels	Phase 5
34	One-click safe fixes	Phase 5
35	App-level event bus	Phase 1
36	Page registry	Phase 1
37	Page lifecycle hooks	Phase 1
38	Operational Settings page	Phase 2
39	Release readiness page	Phase 12

Phase 0 — Current-State Audit and Safety Baseline

Goal

Before making more changes, Codex should document the current state of the branch and confirm that the app is safe to modify.

This phase is mostly inspection and guardrail work. It prevents later phases from accidentally breaking existing workflows while trying to make the app smarter.

Why This Phase Matters

The app already has many moving pieces:

- EOAT audit form
- Master tracker workbook writing
- Compatibility rows
- Robot-side information workflows
- Annotation database
- Notes and tags
- Scheduled daily and weekly summaries

- Dashboard cache
- Workbook validation
- Demo project mode
- NDA-safe repo boundaries

That means the first move should be making sure Codex understands what exists now. Otherwise it may “simplify” something critical, which is software’s polite way of saying “delete the ladder while you are still on the roof.”

Included Features

This phase does not implement a numbered feature directly. It supports every later phase.

Required Work

0.1 Inventory Current Modules

Codex should produce a short architecture inventory covering:

- `app/` pages
- `core/` modules
- `tools/` scripts
- data templates
- demo project files
- tests
- scheduled report scripts
- workbook/annotation services

0.2 Identify Current Entry Points

Document how the app currently starts and which scripts are expected to run standalone.

Include:

- GUI app entry point
- scheduled report scripts
- workbook validation script
- dashboard refresh paths
- any report-generation scripts

0.3 Establish Safety Baseline

Confirm the repo does not contain obvious private project data.

Checks should include:

- no real workbooks committed

- no real reports committed
- no real photos committed
- no local config committed
- no private network paths committed unless intentionally documented as examples
- no customer, part, mold, or internal operational data committed

0.4 Run Existing Tests

Before changing code, Codex should run the current test suite and save the output summary.

0.5 Create Baseline Smoke Checklist

Create or update a manual smoke-test checklist for:

- app launch
- home page load
- audit page load
- load existing audit
- save audit
- compatibility update
- scheduled report status
- workbook validation
- notes/tags page load
- dashboard refresh

Deliverables

- Current architecture inventory
- Current test status summary
- Manual smoke-test checklist
- List of known risks before development

Acceptance Criteria

- Existing behavior is documented before major refactoring
- Tests are run before changes
- Known failures are recorded rather than hidden
- No private project data is added to the repo

Phase 1 — App Architecture Foundation

Goal

Create a cleaner internal app structure so future features can plug in without every page manually knowing about every other page.

This phase should make the app easier to extend, easier to test, and less likely to break when adding new pages.

Included Features

- **#35 App-level event bus**
- **#36 Page registry**
- **#37 Page lifecycle hooks**

Why This Phase Comes First

Several planned features need pages to refresh when data changes:

- Audit saved
- Annotation changed
- Workbook validated
- Report generated
- Project root changed
- Compatibility rows regenerated
- Cache invalidated

Without a central event system and lifecycle hooks, every page will start directly calling every other page. That becomes fragile very quickly. It is the UI equivalent of running pneumatic tubing through a ceiling fan.

Required Work

1.1 Add Page Registry

Create a central page registry that defines pages in one place.

Each page entry should include:

- page key
- display label
- navigation section
- factory/import path
- icon name if applicable
- whether page requires a project root
- whether page should refresh on show
- events the page cares about

Example conceptual model:

```
@dataclass
class PageSpec:
    key: str
```

```
label: str
section: str
factory: Callable
requires_project_root: bool = True
refresh_on_show: bool = False
listens_to: tuple[str, ...] = ()
```

1.2 Replace Hardcoded Page Factory

The dashboard should build navigation and page instances from the registry instead of a long chain of manual page imports and conditional logic.

Expected benefits:

- easier to add pages
- fewer duplicate labels
- cleaner navigation grouping
- easier testing
- less chance of breaking unrelated pages

1.3 Add Page Lifecycle Hooks

Pages should optionally support:

```
on_show()
on_hide()
on_project_root_changed(config)
on_event(event)
can_close() -> bool
```

The dashboard should call these hooks safely if a page implements them.

1.4 Add App Event Bus

Add a simple event bus for app-wide updates.

Suggested event names:

- AuditSaved
- AuditLoaded
- AnnotationChanged
- TagChanged
- TagColorSynced
- RobotInfoUpdated
- CompatibilityRegenerated

- WorkbookValidated
- ReportGenerated
- DashboardCacheInvalidated
- ProjectRootChanged
- SettingsChanged

Each event should include:

- event type
- timestamp
- source
- optional payload

1.5 Update Key Pages to Use Events

At minimum, update:

- Home page
- Audit page
- Audit Progress page
- Workbook Health page
- Notes page
- Tags page
- Scheduled Reports page

These pages should refresh through lifecycle hooks and events instead of scattered manual refresh calls.

Deliverables

- Page registry module
- Event bus module
- Page lifecycle hook support
- Updated dashboard shell
- Tests for registry loading
- Tests for event dispatch

Acceptance Criteria

- Dashboard navigation still works
- Existing pages still open
- Pages can refresh on show
- Pages can respond to events
- Adding a new page requires editing only the registry and page file
- No business logic is moved into the dashboard shell

Phase 2 — Audit Workflow Stabilization

Goal

Make the audit workflow safer, cleaner, and more maintainable before adding more intelligence on top of it.

This phase protects the core workflow: entering audits, loading existing audits, saving edits, managing defaults, preventing accidental loss, and understanding compatibility impacts.

Included Features

- **#5 Dirty-form protection**
- **#6 Split AuditPage into smaller pieces**
- **#7 Configurable defaults**
- **#20 Field history and audit revision history, initial foundation**
- **#29 Compatibility impact preview**
- **#38 Operational Settings page, initial version**

Why This Phase Matters

The audit page is the heart of the app. If the audit page is unstable, every other feature becomes decorative nonsense taped to a cracked foundation.

Before adding Audit Coach, open-item workflows, validation repairs, or evidence tracking, the audit page should be split into smaller, testable parts.

Required Work

2.1 Split the Audit Page

Refactor the current audit page into smaller files and responsibilities.

Recommended structure:

```
app/pages/audit/  
├─ page.py  
├─ audit_form_view.py  
├─ audit_form_controller.py  
├─ audit_form_model.py  
├─ audit_visibility_controller.py  
├─ audit_defaults_controller.py  
├─ machine_lookup_controller.py  
├─ compatibility_panel.py  
└─ interview_panel.py
```

```
├─ annotation_field_adornments.py
├─ audit_save_workflow.py
└─ audit_history_panel.py
```

Move business logic into `core/audit/` where possible:

```
core/audit/
├─ rules.py
├─ defaults.py
├─ validation.py
├─ save_workflow.py
├─ compatibility_autorun.py
├─ compatibility_preview.py
├─ machine_lookup.py
├─ history.py
└─ completion.py
```

2.2 Preserve Current Behavior During Refactor

This phase should not redesign the form yet. It should keep the existing user-facing behavior mostly unchanged while making the code cleaner internally.

Preserve:

- save audit behavior
- load existing audit behavior
- empty-only mode
- generated audit IDs
- machine-number lookup
- field visibility rules
- compatibility update behavior
- annotation field indicators
- robot-side info handoff behavior, where applicable

2.3 Add Dirty-Form Protection

The app should track unsaved changes in the audit form.

Required behavior:

- Detect when a loaded/new audit has unsaved changes
- Warn before loading a different audit
- Warn before clearing the form
- Warn before navigating away if changes would be lost
- Support discard changes

- Support save before leaving

2.4 Add Local Draft Recovery

Add a local ignored draft file for unsaved audit entries.

Suggested behavior:

- Save draft periodically after edits
- Save draft before risky navigation
- Restore draft after app restart if present
- Let user discard draft
- Show draft timestamp

The draft must stay local and must not be committed.

2.5 Move Defaults Into Config

Move hardcoded audit defaults into configuration/reference data.

Examples:

- default auditor
- default plant/area
- default cleanroom state
- quick disconnects present default
- pneumatic quick disconnect type default
- vacuum generator type default
- connection-type changeover difficulty defaults
- documentation/photo defaults
- sensor/vacuum confirmation defaults

The current defaults should remain the initial values.

2.6 Add Operational Settings Page, Initial Version

Create a Settings page for operational defaults.

Initial sections:

- Project root
- Audit defaults
- Connection defaults
- Scheduled report defaults, read-only for now if needed
- Backup behavior, read-only for now if needed

2.7 Add Compatibility Impact Preview

Before saving changes to an existing audit that has compatibility rows, show what will be affected.

The preview should include:

- number of compatible rows tied to this audit
- fields likely to propagate
- whether compatibility rows will be regenerated
- whether generated press views may be refreshed

The user should not have to guess whether editing one source audit will affect 15 rows. That is how spreadsheets develop villain arcs.

2.8 Add Initial Audit History Foundation

Start capturing audit revision history.

At minimum, record:

- audit ID
- timestamp
- action type
- changed fields
- old values
- new values
- source page/tool

This can begin as JSONL or SQLite depending on the later storage strategy.

Deliverables

- Refactored audit page modules
- Core audit workflow modules
- Dirty-form protection
- Draft recovery
- Config-driven defaults
- Initial Settings page
- Compatibility impact preview
- Initial audit history logging

Acceptance Criteria

- Existing audit workflows still pass tests
- User cannot accidentally lose unsaved audit edits without warning
- Defaults are no longer scattered through UI code
- Compatibility impacts are previewed before risky saves

- Audit history records meaningful changes
 - No real data is written into repo files
-

Phase 3 — Audit Coach and Guided Completion

Goal

Turn the audit page from a large form into a guided workflow that tells the user what still matters, what is missing, and why.

This phase makes the app faster and more useful during real audits.

Included Features

- **#1 Audit Coach Mode**
- **#4 Finish This Audit button**
- **#10 “Why is this field hidden?” explanations**

Why This Phase Matters

The audit form already has applicability logic. The next step is exposing that logic to the user.

Instead of forcing the user to hunt through fields manually, the app should say:

- what section is incomplete
- what field should be filled next
- which hidden fields do not apply
- what missing values are actually important
- what can be marked as follow-up

The form should guide the user like a competent technician, not stare blankly like a vending machine with a clipboard.

Required Work

3.1 Add Audit Coach Panel

Add a side panel or collapsible panel on the Audit page.

It should show:

- current audit status
- applicable field completion
- section-by-section completion

- missing applicable fields
- high-priority warnings
- hybrid EOAT warnings
- semantic data warnings
- hidden field explanations
- suggested next action

3.2 Add Section Completion Model

Create a reusable model for audit completion.

For each section, calculate:

- total visible/applicable fields
- completed fields
- missing fields
- fields marked unknown
- fields with follow-up tags
- warning count

Sections should align with the audit form tabs or groups.

3.3 Add “Next Best Field” Logic

The Audit Coach should identify the next most important field to complete.

Priority order should roughly be:

1. required identity fields
2. EOAT type and tooling fields
3. fields that control visibility of other fields
4. major engineering fields
5. sensor/pneumatic/electrical details
6. documentation/photo evidence fields
7. notes and optional fields

3.4 Add “Finish This Audit” Workflow

Add a button that enters a guided completion mode.

Behavior:

- show only missing applicable fields or jump to them one at a time
- explain why each field matters
- allow user to fill value
- allow user to mark Unknown / Not Checked
- allow user to create follow-up
- allow user to tag as Needs Review

- update completion score live

3.5 Add Hidden Field Explanation Viewer

When fields disappear because they do not apply, the user should be able to view why.

Example explanations:

- Vacuum fields hidden because EOAT Type is Mechanical / Gripper
- Electrical detail fields hidden because Electrical/Wiring Present? is No
- Sensor model fields hidden because Sensors Present? is No
- Quick disconnect detail hidden because Quick Disconnects Present? is No

3.6 Avoid False Completeness

If a missing field is intentionally marked unknown or follow-up, the audit should distinguish between:

- complete and verified
- complete with follow-up
- incomplete
- blocked

Do not pretend unknown data is the same as verified data. That is how dashboards become professional-looking fiction.

Deliverables

- Audit Coach panel
- Audit completion model
- Finish This Audit workflow
- Hidden field explanation viewer
- Tests for applicability and completion scoring

Acceptance Criteria

- User can quickly see what is missing from an audit
 - Hidden fields are explainable
 - Audit completion uses applicability rules, not raw blank-cell counting
 - Guided completion does not overwrite user data
 - Follow-up/unknown fields are represented honestly
-

Phase 4 — Annotations, Notes, Tags, and Open Items

Goal

Make annotations, notes, tags, validation warnings, and follow-ups work together as one action-management system.

This phase turns the annotation system into a practical work queue.

Included Features

- #2 Apply/reject annotation suggestions
- #3 Unified Open Items / Action Board

Why This Phase Matters

Tags and notes are only useful if they lead to action. Otherwise they become colorful decorations on top of unresolved work, which is basically engineering confetti.

The app should collect all open concerns into one place so the user can decide what needs attention next.

Required Work

4.1 Improve Annotation Suggestions UI

Replace text-only suggestion output with a structured table.

Suggested columns:

- Apply checkbox
- Tag
- Target field
- Reason
- Severity or confidence
- Suggested comment
- Existing note/tag status

Buttons:

- Apply Selected
- Apply All High Confidence
- Ignore Selected
- Create Follow-Up Actions

4.2 Track Ignored Suggestions

If the user ignores a suggestion, the app should avoid repeatedly showing the same suggestion unless the underlying data changes.

Ignored suggestion records should include:

- suggestion ID or hash
- audit ID
- target field
- reason
- ignored timestamp
- data fingerprint if practical

4.3 Add Unified Open Items Board

Create a page that shows all unresolved project concerns.

Sources should include:

- field tags
- notes marked open
- unresolved follow-ups
- validation findings
- missing evidence warnings
- compatibility concerns
- documentation gaps
- upcoming due dates

4.4 Add Open Item Statuses

Suggested statuses:

- Open
- In Progress
- Waiting on Info
- Blocked
- Resolved
- Dismissed

4.5 Add Open Item Actions

Each open item should support:

- open target audit
- jump to field
- add note
- assign owner if owner support exists

- set due date
- mark resolved
- dismiss with reason
- export to report

4.6 Add Summary Metrics

The board should show:

- total open items
- critical open items
- overdue follow-ups
- missing evidence count
- data conflict count
- items resolved this week

Deliverables

- Annotation suggestions table
- Apply/ignore suggestion workflow
- Open Items page
- Open item status model
- Tests for suggestion application and open item aggregation

Acceptance Criteria

- Suggestions can be applied without manually tagging fields one at a time
- Ignored suggestions do not nag forever without reason
- Open items from multiple systems appear in one place
- Open items can be resolved or dismissed with traceability
- Audit field indicators update after tag changes

Phase 5 — Workbook Validation and Repair System

Goal

Upgrade workbook validation from a text warning report into a structured, repairable health system.

This phase makes workbook problems actionable.

Included Features

- **#9 Repairable validation findings**
- **#20 Field history and audit revision history, expanded**
- **#22 Local SQLite audit mirror, foundation decision/initial implementation**
- **#30 Physical vs compatible row consistency checks**
- **#32 Workbook lock detector**
- **#33 Validation severity levels**
- **#34 One-click safe fixes**

Why This Phase Matters

The workbook is one of the most important deliverables, but Excel is not a reliable app database by itself. Validation should identify real problems, explain severity, and offer safe repair paths where possible.

This phase makes the system more truthful and safer.

Required Work

5.1 Create Structured Validation Finding Model

Introduce a validation finding object.

Suggested fields:

- finding ID
- severity
- category
- sheet name
- row number
- column name
- audit ID
- machine number
- message
- recommended action
- fix available
- fix ID if available
- source validator

Suggested severities:

- Blocker
- Error
- Warning
- Info
- Auto-fixable

5.2 Preserve Existing Markdown Output

Do not remove existing human-readable validation reports.

Instead, add structured outputs:

- JSON validation findings
- table view in UI
- optional CSV export

5.3 Add Workbook Health Table

Workbook Health page should show findings in a table with filters:

- severity
- sheet
- field
- audit ID
- category
- fix available

Actions:

- open audit
- jump to field if possible
- create annotation
- create follow-up
- apply safe fix
- export report

5.4 Add Physical vs Compatible Row Checks

Validate relationships between source physical audits and compatibility rows.

Check for:

- compatible rows with missing source audit
- stale compatible rows
- extra compatible rows
- missing compatible rows
- incompatible inherited values
- fields that should not be manually edited in compatible rows

5.5 Add Workbook Lock Detector

Before writing to workbooks, detect whether the workbook is likely locked/open.

If locked:

- show clear message
- do not silently fail
- offer retry
- queue operation if safe
- explain which workbook is locked

5.6 Add One-Click Safe Fixes

Only implement fixes that are objectively safe.

Examples:

- clear stale hidden non-applicable fields to N/A
- repair known legacy headers
- refresh generated views
- reapply formatting
- rebuild dropdown data validation

Each fix should:

1. create backup
2. preview change count
3. apply fix
4. rerun validation
5. report before/after result

5.7 Expand Audit History

Audit revision history should connect to validation and repair actions.

Record:

- user-initiated saves
- validation auto-fixes
- compatibility regenerations
- workbook repair actions
- tag color sync changes if practical

5.8 Decide on SQLite Audit Mirror

This phase should at least decide and document whether to add a local SQLite mirror of audit data.

If implemented now, start small:

- mirror audit IDs and key fields
- mirror compatibility relationships

- mirror validation findings
- mirror save history

The Excel workbook can remain the primary deliverable, but SQLite can improve speed, search, history, and repair tracking.

Deliverables

- Structured validation findings model
- JSON validation output
- Workbook Health findings table
- Severity classification
- Lock detection
- Safe auto-fix framework
- Physical/compatible consistency checks
- Expanded audit history
- SQLite mirror decision or initial implementation

Acceptance Criteria

- Validation findings are structured and filterable
- Existing reports still exist
- Safe fixes create backups
- Workbook lock failures are clear
- Compatibility row issues are detectable
- Repair actions are traceable

Phase 6 — Dashboard Performance, Cache, and Telemetry

Goal

Make the app faster, more transparent, and easier to diagnose when something is slow.

This phase improves the user experience without changing the engineering workflow too much.

Included Features

- **#13 Cache staleness improvements**
- **#14 Structured performance logging**
- **#22 Local SQLite audit mirror, performance use if implemented**

Why This Phase Matters

The app should open quickly and let the user start working without waiting for every workbook and report to be scanned deeply.

Slow dashboards train users not to trust them. Fast dashboards feel alive. Also, less waiting means fewer opportunities for the user to question every life choice that led to watching a progress spinner.

Required Work

6.1 Expand Cache Source Tracking

The dashboard cache should track all sources that affect dashboard metrics.

Include:

- master tracker workbook
- activity log
- scheduled report folders
- daily/weekly report folders
- task/schedule files
- annotation database
- Robot Info workbook if not handled elsewhere
- validation finding exports
- photo index files
- documentation gap outputs

6.2 Add “Why Cache Is Stale” Explanation

When cache is stale, show why.

Example:

```
Dashboard cache stale because:  
- annotations.sqlite changed 2 minutes ago  
- EOAT_Master_Tracker.xlsx changed 5 minutes ago  
- Weekly_Status_Reports folder changed today
```

6.3 Preserve Quick Refresh vs Deep Refresh

Keep two separate refresh levels:

- Quick Refresh: use cache or lightweight metadata
- Deep Refresh: rescan source files and rebuild metrics

The UI should make the difference clear.

6.4 Add Structured Performance Logging

Move from plain text performance logs to JSONL or structured records.

Fields should include:

- timestamp
- operation name
- duration seconds
- success/failure
- source page/tool
- project root mode
- source file count if relevant
- warning/error summary

6.5 Add Performance Page

Create a page showing:

- slowest operations
- recent app startup duration
- recent dashboard refresh durations
- report generation duration
- workbook validation duration
- audit save duration
- cache hit/miss counts

6.6 Add Performance Regression Tests

Where practical, add tests or smoke checks that guard against major slowdowns.

Example thresholds can be loose at first:

- dashboard shell loads without deep scan
- quick refresh does not open every workbook
- cache read is fast with demo data

Deliverables

- Expanded cache metadata tracking
- Cache stale-reason explanations
- Structured performance log
- Performance diagnostics page
- Performance regression tests/smoke checks

Acceptance Criteria

- App can show cached dashboard data quickly
 - User can tell why a cache is stale
 - Deep refresh remains available but intentional
 - Slow operations are recorded in structured form
 - Performance page shows useful diagnostics
-

Phase 7 — Scheduled Reports and Report Reliability

Goal

Make automated daily and weekly reports visible, testable, recoverable, and genuinely useful.

Included Features

- **#15 Scheduled report calendar preview**
- **#16 Catch-up summaries**
- **#17 Smarter scheduled report content**
- **#18 Task Scheduler preflight**

Why This Phase Matters

Scheduled reports should not be invisible magic. They should be inspectable and testable.

The app should answer:

- Are the tasks installed?
- Will they run tonight?
- What happens if one was missed?
- What data will the report use?
- Where did the report go?
- Did it succeed?

Invisible automation is just a gremlin with a wristwatch.

Required Work

7.1 Add Scheduled Reports Status Page Upgrade

The page should show:

- daily task installed status
- weekly task installed status
- last run status
- last generated daily report
- last generated weekly report
- next expected run
- missed report dates
- log file locations

7.2 Add Calendar Preview

Show the next 14 or 30 days of expected scheduled report behavior.

For each day:

- date
- expected report type
- scheduled time
- already exists / due / skipped / not scheduled
- whether it is a workday

7.3 Add Catch-Up Summary Workflow

If reports were missed, allow the user to generate catch-up reports.

Actions:

- generate selected missed daily reports
- generate selected missed weekly reports
- mark selected dates intentionally skipped
- open generated reports

7.4 Add Task Scheduler Preflight

Check:

- Windows platform
- PowerShell available
- Task Scheduler accessible
- daily task exists
- weekly task exists

- scripts exist
- Python executable exists
- project root exists
- output folders writable
- last scheduled log readable

7.5 Add Manual Dry Runs

Buttons:

- Run Daily Dry Run
- Run Weekly Dry Run
- Generate Daily Now
- Generate Weekly Now
- Install/Repair Tasks
- Open Logs
- Copy Diagnostics

7.6 Improve Report Content Inputs

Scheduled reports should use real app activity where available.

Potential sources:

- activity log
- completed tasks
- audit saves
- annotations created
- validation findings
- open items
- reports generated
- schedule progress
- blockers
- changed files

The reports should avoid generic filler if real project activity exists.

Deliverables

- Scheduled report calendar preview
- Catch-up workflow
- Task Scheduler preflight panel
- Dry-run actions
- Better scheduled report data collection

Acceptance Criteria

- User can see whether scheduled reports are actually installed
 - User can see upcoming scheduled runs
 - Missed reports are visible and recoverable
 - Manual dry runs are possible
 - Reports include real project activity when available
-

Phase 8 — Search, Navigation, and Productivity Layer

Goal

Make the app easier to navigate as the number of pages, audits, reports, notes, and findings grows.

Included Features

- #11 Global command palette
- #12 Global search
- #23 SQLite FTS search
- #31 App-native Press View, navigation side

Why This Phase Matters

Once the app has many pages and records, clicking through menus becomes slow. The user should be able to jump directly to the thing they need.

This phase turns the app from a collection of pages into a fast workbench.

Required Work

8.1 Add Global Command Palette

Add a shortcut such as `Ctrl+K`.

Supported commands should include:

- open audit by ID
- open machine/press
- run workbook validation
- generate daily summary
- generate weekly summary

- open latest report
- deep refresh dashboard
- open notes by tag
- open settings
- open scheduled report diagnostics

8.2 Add Command Registry

Commands should be registered centrally.

Each command should include:

- command ID
- display name
- search aliases
- required page/tool
- handler function
- safety level
- whether it requires confirmation

8.3 Add Global Search

Search should cover:

- audits
- machines/presses
- notes
- tags
- open items
- validation findings
- reports
- photo index records
- standards/docs records if available

8.4 Add Search Filters

Filters:

- type
- audit ID
- machine/press
- tag
- status
- severity
- date
- due date

8.5 Add SQLite FTS if Appropriate

If notes/audits are mirrored to SQLite, add FTS search.

Search operators can eventually include:

```
tag:"Needs Review"
machine:12
audit:AUD-2026
status:open
due:<2026-06-01
```

8.6 Add App-Native Press View

Create a Press View page or search result type.

Each press should show:

- physical audits
- compatible tools
- open items
- validation warnings
- photo coverage
- pilot candidacy
- last updated date

This does not replace Excel grouping. It gives the app a faster operational view.

Deliverables

- Command palette
- Command registry
- Global search page/dialog
- Search index or SQLite FTS implementation
- App-native Press View

Acceptance Criteria

- User can jump to important pages/tools quickly
 - Search returns audits, notes, tags, findings, and reports
 - Press View gives a clear summary by machine/press
 - Commands requiring destructive actions ask for confirmation
-

Phase 9 — Evidence, Photos, PM, BOM, and Documentation Coverage

Goal

Improve project evidence quality and documentation completeness.

This phase supports audit defensibility, PM checklist generation, spare parts standardization, and final documentation coverage.

Included Features

- **#8 Gripper presets mini parts database**
- **#24 Photo evidence coverage**
- **#25 Phone-friendly photo intake**

Why This Phase Matters

The project is not just about filling out cells. It needs evidence, standards, repeatable documentation, and maintenance support.

Photos, PM checklist coverage, and BOM/spare parts records make the final project more useful to Manufacturing Engineering, Maintenance, Quality, and leadership.

Required Work

9.1 Add Photo Evidence Categories

Define expected photo categories per EOAT type.

Possible categories:

- overall EOAT
- robot connection
- EOAT-side pneumatic circuits
- sensors
- quick disconnects
- tubing routing
- grippers
- vacuum cups
- mounting hardware
- cable management
- wear/damage
- process binder reference

9.2 Add Evidence Coverage Per Audit

Each audit should show photo coverage.

Example statuses:

- complete
- partial
- missing
- not applicable
- follow-up needed

9.3 Add Evidence Validation Warnings

Validation should warn when:

- audit is marked complete but required evidence is missing
- pilot candidate lacks before photos
- issue has no supporting evidence
- documentation marked complete but no document/photo link exists

9.4 Add Phone-Friendly Photo Intake Workflow

At minimum:

- create audit-specific intake folder
- show folder path
- generate photo checklist
- allow user to import/associate photos later

Optional later version:

- local web upload page
- QR code for phone upload
- same-network upload support

9.5 Add Gripper Preset Database

Turn gripper model mappings into a small reference database.

Each preset can include:

- friendly name
- part number
- manufacturer
- type
- default pressure/vacuum logic if relevant
- notes

- status

9.6 Add PM/BOM Coverage Hooks

This phase does not need to fully build the PM/BOM pages if those already exist, but it should create the data hooks needed for:

- PM checklist generation
- spare parts review
- missing BOM detection
- standard part opportunities

Deliverables

- Photo evidence category model
- Evidence coverage UI per audit
- Evidence validation checks
- Photo intake workflow
- Gripper preset reference data
- PM/BOM coverage hooks

Acceptance Criteria

- Each audit can show evidence completeness
- Missing evidence is visible and actionable
- Photo intake is less manual
- Gripper presets are not hardcoded only in Python
- PM/BOM workflows can reuse evidence and reference data

Phase 10 — Standards, FMEA, Pilot, and Engineering Analysis

Goal

Turn collected audit data into engineering analysis and improvement recommendations.

This phase helps the app support standardization, FMEA, pilot selection, and performance improvement work.

Included Features

- **#26 Pilot candidate evidence packet, analysis side**
- **#27 FMEA generation from tags and issues**

- **#28 Standards compliance score**
- **#31 App-native Press View, analysis side**

Why This Phase Matters

The app should not only collect data. It should help answer engineering questions:

- Which EOATs are highest risk?
- Which standards are not being met?
- Which failure modes are most common?
- Which pilot candidate has the strongest evidence?
- What should be improved first?

This phase turns the project from “database-building” into “engineering decision support.” Fancy phrase. Actually useful.

Required Work

10.1 Add Standards Compliance Scoring

Create a scoring model based on EOAT standards.

Categories may include:

- pneumatic routing
- sensor standards
- quick disconnect standards
- cable management
- mechanical mounting
- weight/reach concerns
- safety concerns
- documentation completeness
- PM readiness
- BOM/spare parts readiness

Scores should distinguish:

- compliant
- warning
- fail
- not applicable
- unknown/follow-up

10.2 Show Compliance Per Audit

Each audit should show:

- overall compliance score
- category scores
- failed standards
- missing evidence
- recommended actions

10.3 Add Compliance Summary by Press/Cell

Press View should show compliance rollups:

- average compliance score
- worst category
- open standards issues
- pilot candidate relevance

10.4 Generate FMEA Suggestions from Real Data

FMEA suggestions should use:

- issue categories
- known issues
- validation findings
- tags
- notes
- evidence gaps
- sensor/pneumatic/mechanical fields

Suggested failure modes may include:

- vacuum loss
- misalignment
- tubing failure
- sensor failure
- mechanical wear
- quick disconnect mismatch
- documentation failure
- maintenance access issue

The user should approve or edit severity, occurrence/frequency, and detection ratings.

10.5 Add Pilot Candidate Evidence Packet

For each pilot candidate, generate an evidence-backed summary.

Include:

- machine/press
- EOAT type
- known issues
- failure modes
- standards gaps
- downtime/scrap/cycle-time concerns if available
- photos/evidence coverage
- expected improvement area
- implementation difficulty
- risks
- recommended next action

10.6 Add Analysis Exports

Export:

- compliance summary
- FMEA draft
- top failure modes
- pilot candidate packet
- press/cell risk summary

Deliverables

- Standards compliance model
- Compliance UI
- FMEA suggestion system
- Pilot candidate evidence packet generator
- Analysis exports
- Press View analysis rollups

Acceptance Criteria

- Standards scoring uses real audit data
 - Unknown and not-applicable fields are treated honestly
 - FMEA suggestions are reviewable, not auto-final
 - Pilot packets are evidence-backed
 - Analysis outputs can support leadership review
-

Phase 11 — Final Handoff and Leadership-Ready Outputs

Goal

Prepare the app to generate clean final deliverables for the end of the project.

Included Features

- #26 Pilot candidate evidence packet, final output side

Why This Phase Matters

A strong final project needs more than working code. It needs clean deliverables that leadership, maintenance, manufacturing engineering, and future users can actually use.

This phase turns app data into polished project outputs.

Required Work

11.1 Add Final Handoff Builder

The Final Handoff page should collect and package:

- EOAT database summary
- standards guideline status
- PM checklist package
- FMEA output
- KPI dashboard/export
- pilot result summary
- training/work instruction materials
- documentation gap summary
- open items carryover
- recommendations
- executive summary

11.2 Add Final Deliverable Checklist

The app should show final deliverable readiness.

Example:

Final Handoff Readiness

EOAT Database: Ready

Standards Guidelines: Draft

PM Checklist: Ready

FMEA: Needs Review

Pilot Results: In Progress

Training Materials: Missing

Executive Summary: Not Started

11.3 Add Leadership Summary Export

Generate a short leadership-friendly summary.

It should include:

- project objective
- work completed
- major findings
- pilot recommendation/results
- KPI impact if available
- remaining risks
- next steps

11.4 Add Technical Appendix Export

Generate a more detailed technical appendix for engineering/maintenance.

Include:

- validation findings
- standards gaps
- FMEA details
- audit coverage
- PM/BOM findings
- photos/evidence references
- open items

11.5 Add Handoff Package Folder Builder

Create a dated output folder with predictable structure.

Example:

```
06_Final_Handoff/  
└─ Final_Handoff_Package_YYYYMMDD/  
   ├── Executive_Summary.md  
   ├── Technical_Appendix.md  
   ├── FMEA_Output.xlsx  
   ├── KPI_Summary.xlsx  
   ├── PM_Checklists/  
   ├── Pilot_Candidates/  
   ├── Standards/  
   └─ Open_Items_Carryover.md
```

Deliverables

- Final Handoff page upgrade
- Final deliverable readiness checklist
- Leadership summary export
- Technical appendix export
- Final handoff package builder

Acceptance Criteria

- Final handoff readiness is visible
- Outputs are saved in predictable folders
- Executive and technical outputs are separated
- Open items are not hidden
- Final package can be regenerated safely without overwriting prior packages

Phase 12 — Repo Safety, Release Readiness, and Long-Term Maintenance

Goal

Make the project safer to maintain, safer to push to GitHub, and easier to hand off or continue later.

Included Features

- **#21 Backup retention cleanup**
- **#39 Release readiness page**

Why This Phase Matters

The app is handling project files, workbooks, generated reports, local configs, backups, and potentially sensitive company data. The repo should remain safe and clean.

Also, backups should not grow forever until they become a second project by accident.

Required Work

12.1 Add Backup Manager

Create a backup management system for generated workbook backups.

It should show:

- backup count
- total backup size
- oldest backup
- newest backup
- backups by source workbook
- cleanup recommendation

12.2 Add Backup Retention Policy

Suggested default policy:

- keep all backups from last 7 days
- keep last 25 backups per workbook
- keep milestone backups
- never delete backups if latest validation has blockers
- require confirmation before cleanup

12.3 Add Release Readiness Page

Create a page that checks whether the repo/app is ready for commit or release.

Checks:

- tests pass
- app smoke test pass if available
- repo safety audit pass
- no real workbooks staged
- no real photos staged
- no local config staged
- no generated reports staged unless intentionally allowed
- README/USAGE up to date
- demo data still works

- branch status visible

12.4 Add Staged File Safety Scanner

Scan staged files or working tree files for unsafe content.

Flag:

- `.xlsx` files outside allowed demo/template folders
- private network paths
- real project root paths
- generated status reports
- local config files
- logs/cache files
- photos
- possible customer/mold/part identifiers if patterns are known

12.5 Add Pre-Commit Hook Option

Allow the user to install a local pre-commit hook that runs the safety scanner before commits.

The hook should block obvious unsafe commits.

12.6 Add Maintenance Documentation

Update documentation for:

- adding new pages
- adding new tools
- adding validation rules
- adding settings
- adding report generators
- running tests
- preparing a GitHub commit safely

Deliverables

- Backup Manager
- Backup retention cleanup
- Release Readiness page
- Staged file safety scanner
- Optional pre-commit hook installer
- Updated maintenance documentation

Acceptance Criteria

- User can see and clean old backups safely

- Release readiness is visible from inside app
 - Unsafe repo content is flagged before push
 - Demo mode remains safe
 - Future development is documented
-

Cross-Phase Rules

These rules apply to every phase.

1. No Real Company Data in Repo

Do not commit:

- real workbooks
- real generated reports
- real photos
- local config files
- private paths
- mold/part/customer data
- machine-specific confidential data

Use demo data and templates only.

2. Preserve CLI Tools

If a feature has an existing command-line tool, do not break it.

The app should be a UI around modular tools, not the only way to run them.

3. Keep Business Logic Out of UI Files

UI files should coordinate and display. Core modules should decide.

Bad:

AuditPage directly contains every workbook rule, validation rule, default, and save workflow.

Good:

AuditPage calls AuditSaveWorkflow, AuditRules, AuditDefaults, and AnnotationService.

4. Always Backup Before Workbook Writes

Any feature that modifies Excel workbooks should create a backup unless the operation is proven harmless and intentionally exempted.

5. Every Destructive Action Needs Confirmation

Examples:

- cleanup backups
- apply auto-fixes
- discard draft
- overwrite report package
- regenerate compatibility rows
- clear stale values

6. Unknown Is Not the Same as N/A

The app should distinguish:

- N/A because field does not physically apply
- Unknown because the auditor did not check
- Blank because it was missed
- Follow-up because it needs later review
- Verified because it was actually observed

This is one of the most important truth rules in the whole app.

7. Validation Must Be Explainable

Every warning should explain:

- what is wrong
- where it is
- why it matters
- what the user can do next

No vague spreadsheet horoscopes.

8. Cache Must Never Lie

Cached data should be clearly labeled.

The app should show:

- fresh
- stale
- unknown
- deep refresh needed
- source changed

9. Reports Should Be Evidence-Based

Reports should prefer real activity logs, audit saves, open items, validation findings, and completed tasks over generic filler.

10. Tests Are Required for Behavior Changes

Every phase should add or update tests.

Minimum test categories:

- unit tests for core logic
- workflow tests for save/validation/report generation
- smoke tests for pages where practical
- safety tests for repo scanner and config handling

Recommended Codex Execution Strategy

Do Not Ask Codex to Build Everything at Once

Each phase should be given to Codex separately.

A good rhythm:

1. Give Codex one phase.
2. Ask it to inspect relevant files first.
3. Ask it to make a short implementation plan.
4. Ask it to implement in small commits or logical patches.
5. Ask it to run tests.
6. Ask it to summarize changed files, behavior changes, and risks.
7. Review manually before moving on.

If Codex tries to do all twelve phases at once, stop it. That path leads to a 4,000-line refactor and a new religion based around stack traces.

Suggested Codex Prompt Template

You are working in the EOAT Command Center repository.

Implement Phase X from the EOAT Command Center Improvement Roadmap.

Before editing code:

1. Inspect the current files related to this phase.
2. Summarize the current architecture.
3. Identify the smallest safe implementation path.
4. List expected tests.

Implementation requirements:

- Preserve existing behavior unless this phase explicitly changes it.
- Do not commit real company data.
- Keep business logic out of UI files where practical.
- Add tests for new behavior.
- Update documentation if user-facing behavior changes.
- Run the relevant tests before final summary.

Final response requirements:

- List files changed.
- List features implemented.
- List tests run and results.
- List any known risks or follow-up items.

Recommended Milestone Bundles

If this roadmap needs to be compressed into fewer larger milestones, use this grouping.

Milestone A — Architecture and Audit Safety

Includes:

- Phase 1
- Phase 2
- Phase 3 foundation

Outcome:

The app architecture is cleaner, the audit page is safer, and the user cannot easily lose work.

Milestone B — Action Management and Validation

Includes:

- Phase 4
- Phase 5

Outcome:

Warnings, tags, notes, findings, and follow-ups become one actionable system.

Milestone C — Speed and Automation Reliability

Includes:

- Phase 6
- Phase 7

Outcome:

The app is faster, cache behavior is clearer, and scheduled reports become visible/recoverable.

Milestone D — Navigation and Evidence

Includes:

- Phase 8
- Phase 9

Outcome:

The app becomes easier to navigate and audit evidence becomes much more defensible.

Milestone E — Engineering Analysis and Final Handoff

Includes:

- Phase 10
- Phase 11
- Phase 12

Outcome:

The project produces stronger final engineering outputs and is safer to maintain/push.

Practical Priority Recommendation

If time is limited, prioritize in this order:

1. Phase 2 — Audit Workflow Stabilization
2. Phase 3 — Audit Coach and Guided Completion
3. Phase 5 — Workbook Validation and Repair System
4. Phase 7 — Scheduled Reports and Report Reliability
5. Phase 4 — Open Items Board
6. Phase 6 — Performance and Cache
7. Phase 8 — Search and Navigation
8. Phase 9 — Evidence/Photos/PM/BOM
9. Phase 10 — Analysis/FMEA/Pilot
10. Phase 11 — Final Handoff
11. Phase 12 — Release/maintenance safety
12. Phase 1 should happen early, but it can be implemented lightly at first if needed.

The most important practical improvements are:

- protect audit work from accidental loss
- guide the user through missing applicable fields
- make validation findings actionable
- make scheduled reports visible and recoverable
- keep the repo safe from private data

Everything else builds on that.

Final Implementation Philosophy

The EOAT Command Center should become a modular engineering workbench.

It should not merely store audit data. It should help answer:

- What still needs to be audited?
- What information is missing?
- What evidence is missing?
- Which warnings actually matter?
- Which EOATs are risky?
- Which standards are not being met?
- Which pilot candidates are strongest?
- What reports are due?
- What is ready for final handoff?

The target experience is simple:

Open app.
See project status.
Know what matters next.
Click directly into the problem.
Fix or tag it.
Generate reports.
Leave a clean audit trail.

That is the difference between an app that merely exists and an app that actually helps a human survive an engineering internship without becoming one with the spreadsheet.